



## Number Range (medium)

We'll cover the following



- Problem Statement
- Try it yourself
- Solution
- Code
  - Time complexity
  - Space complexity

### Problem Statement

Given an array of numbers sorted in ascending order, find the range of a given number 'key'. The range of the 'key' will be the first and last position of the 'key' in the array.

Write a function to return the range of the 'key'. If the 'key' is not present return [-1, -1].

#### Example 1:

Input: [4, 6, 6, 6, 9], key = 6  
Output: [1, 3]

#### Example 2:

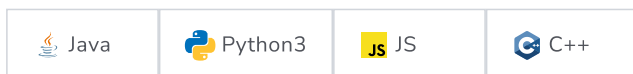
Input: [1, 3, 8, 10, 15], key = 10  
Output: [3, 3]

#### Example 3:

Input: [1, 3, 8, 10, 15], key = 12  
Output: [-1, -1]

### Try it yourself

Try solving this question here:



```
1 class FindRange {
2
3     public static int[] findRange(int[] arr, int key) {
4         int[] result = new int[] { -1, -1 };
5         // TODO: Write your code here
6     }
7 }
```



```

6     return result;
7 }
8
9 public static void main(String[] args) {
10     int[] result = FindRange.findRange(new int[] { 4, 6, 6, 6, 9 }, 6);
11     System.out.println("Range: [" + result[0] + ", " + result[1] + "]");
12     result = FindRange.findRange(new int[] { 1, 3, 8, 10, 15 }, 10);
13     System.out.println("Range: [" + result[0] + ", " + result[1] + "]");
14     result = FindRange.findRange(new int[] { 1, 3, 8, 10, 15 }, 12);
15     System.out.println("Range: [" + result[0] + ", " + result[1] + "]");
16 }
17 }

```

Run

Save

Reset



## Solution

The problem follows the **Binary Search** pattern. Since Binary Search helps us find a number in a sorted array efficiently, we can use a modified version of the Binary Search to find the first and the last position of a number.

We can use a similar approach as discussed in [Order-agnostic Binary Search](#). We will try to search for the 'key' in the given array; if the 'key' is found (i.e. `key == arr[middle]`) we have two options:

1. When trying to find the first position of the 'key', we can update `end = middle - 1` to see if the key is present before `middle`.
2. When trying to find the last position of the 'key', we can update `start = middle + 1` to see if the key is present after `middle`.

In both cases, we will keep track of the last position where we found the 'key'. These positions will be the required range.

## Code

Here is what our algorithm will look like:

```

1 class FindRange {
2
3     public static int[] findRange(int[] arr, int key) {
4         int[] result = new int[] { -1, -1 };
5         result[0] = search(arr, key, false);
6         if (result[0] != -1) // no need to search, if 'key' is not present in the input array
7             result[1] = search(arr, key, true);
8         return result;
9     }
10
11     // modified Binary Search
12     private static int search(int[] arr, int key, boolean findMaxIndex) {
13         int keyIndex = -1;
14         int start = 0, end = arr.length - 1;
15         while (start <= end) {
16             int mid = start + (end - start) / 2;
17             if (key < arr[mid]) {
18                 end = mid - 1;
19             } else if (key > arr[mid]) {
20                 start = mid + 1;
21             } else { // key == arr[mid]
22                 keyIndex = mid;

```



```
22         keyIndex = mid;
23         if (findMaxIndex)
24             start = mid + 1; // search ahead to find the last index of 'key'
25         else
26             end = mid - 1; // search behind to find the first index of 'key'
27     }
28 }
29 return keyIndex;
30 }
31
32 public static void main(String[] args) {
33     int[] result = FindRange.findRange(new int[] { 4, 6, 6, 6, 9 }, 6);
34     System.out.println("Range: [" + result[0] + ", " + result[1] + "]");
35     result = FindRange.findRange(new int[] { 1, 3, 8, 10, 15 }, 10);
36     System.out.println("Range: [" + result[0] + ", " + result[1] + "]");
37     result = FindRange.findRange(new int[] { 1, 3, 8, 10, 15 }, 12);
38     System.out.println("Range: [" + result[0] + ", " + result[1] + "]");
39 }
40 }
```

Run

Save

Reset



## Time complexity

Since, we are reducing the search range by half at every step, this means that the time complexity of our algorithm will be  $O(\log N)$  where 'N' is the total elements in the given array.

## Space complexity

The algorithm runs in constant space  $O(1)$ .



Next Letter (medium)

Search in a Sorted Infinite Array (medium)